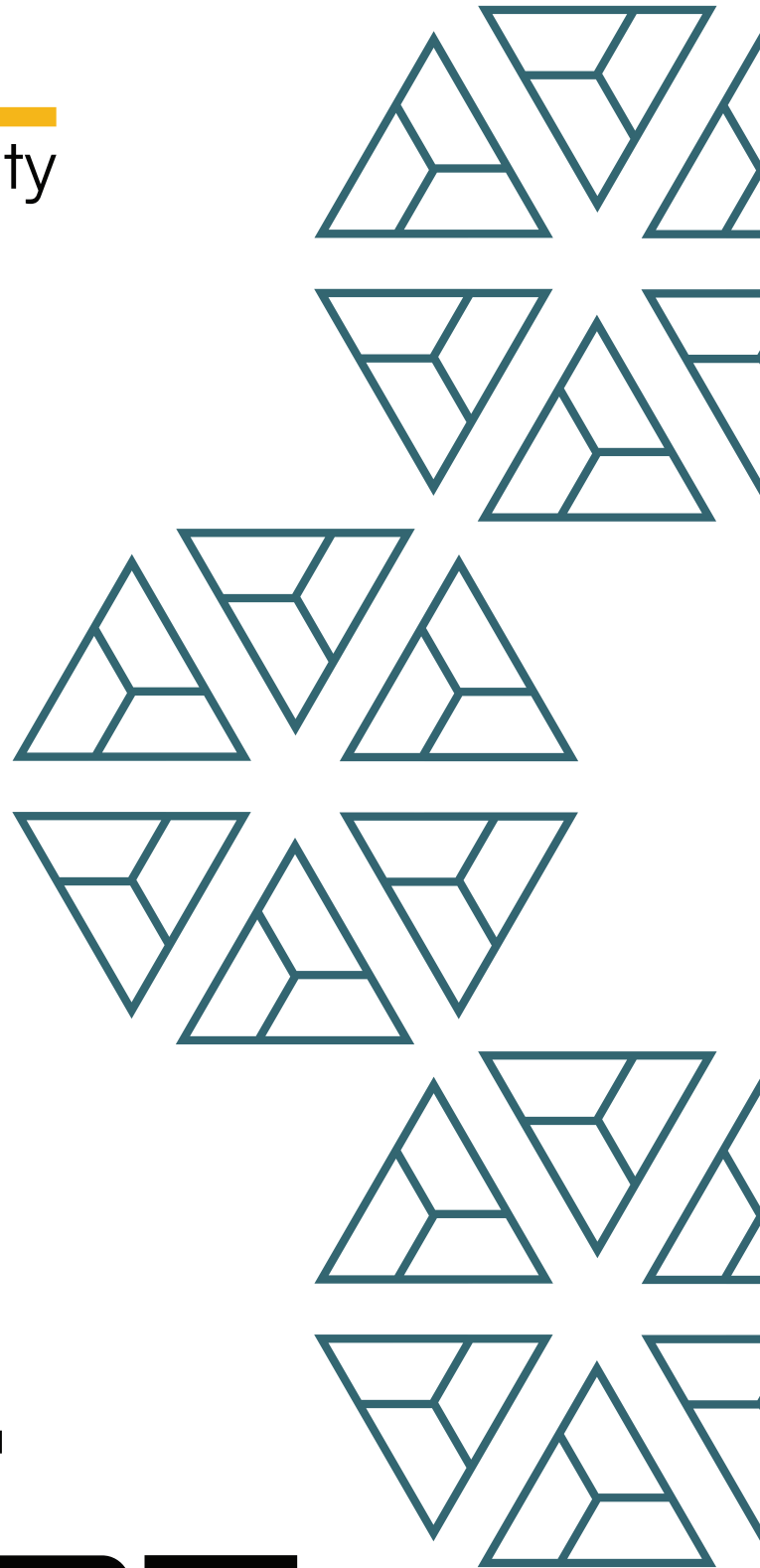




BAIL
security



Lista Dao
LisAster

FINAL REPORT

May '2026

Disclaimer:

Security assessment projects are time-boxed and often reliant on information that may be provided by a client, its affiliates, or its partners. As a result, the findings documented in this report should not be considered a comprehensive list of security issues, flaws, or defects in the target system or codebase.

The content of this assessment is not an investment. The information provided in this report is for general informational purposes only and is not intended as investment, legal, financial, regulatory, or tax advice. The report is based on a limited review of the materials and documentation provided at the time of the audit, and the audit results may not be complete or identify all possible vulnerabilities or issues. The audit is provided on an "as-is," "where-is," and "as-available" basis, and the use of blockchain technology is subject to unknown risks and flaws.

The audit does not constitute an endorsement of any particular project or team, and we make no warranties, expressed or implied, regarding the accuracy, reliability, completeness, or availability of the report, its content, or any associated services or products. We disclaim all warranties, including the implied warranties of merchantability, fitness for a particular purpose, and non-infringement.

We assume no responsibility for any product or service advertised or offered by a third party through the report, any open-source or third-party software, code, libraries, materials, or information linked to, called by, referenced by, or accessible through the report, its content, and the related services and products. We will not be liable for any loss or damages incurred as a result of the use or reliance on the audit report or the smart contract.

The contract owner is responsible for making their own decisions based on the audit report and should seek additional professional advice if needed. The audit firm or individual assumes no liability for any loss or damages incurred as a result of the use or reliance on the audit report or the smart contract. The contract owner agrees to indemnify and hold harmless the audit firm or individual from any and all claims, damages, expenses, or liabilities arising from the use or reliance on the audit report or the smart contract.

By engaging in a smart contract audit, the contract owner acknowledges and agrees to the terms of this disclaimer.

1. Project Details

Important:

Please ensure that the deployed contract matches the source-code of the last commit hash.

Project	Lista Dao – LisAster - Audit Report
Website	lista.org
Language	Solidity
Methods	Manual Analysis
Github repository	https://github.com/lista-dao/lista-new-contracts/tree/7084d49bb14f567f407e720f88bca43744cc00e8/src/lisaster
Resolution 1	https://github.com/lista-dao/lista-new-contracts/tree/a2ee553ca88ecb9f283d670351c0f2c10f6d3d68/src/lisaster
	https://github.com/lista-dao/lista-new-contracts/tree/caa84d144640648b0ba3b6a00595647e8c9d7160/src Please note that this commit changed the reward seeding to Aster instead of LisAster and implements a claimAndStake function which leverages the AsterVault to convert Aster to LisAster

2. Detection Overview

Severity	Found	Resolved	Partially Resolved	Acknowledged (no changes made)	Failed resolution	Open
High	1	1				
Medium	2	2				
Low	7	2		5		
Informational	15	4		11		
Governance	4			4		
Total	29	9		20		

2.1 Detection Definitions

Severity	Description
High	The problem poses a significant threat to the confidentiality of a considerable number of users' sensitive data. It also has the potential to cause severe damage to the client's reputation or result in substantial financial losses for both the client and the affected users.
Medium	While medium level vulnerabilities may not be easy to exploit, they can still have a major impact on the execution of a smart contract. For instance, they may allow public access to critical functions, which could lead to serious consequences.
Low	Poses a very low-level risk to the project or users. Nevertheless the issue should be fixed immediately
Informational	Effects are small and do not post an immediate danger to the project or users
Governance	Governance privileges which can directly result in a loss of funds or other potential undesired behavior

3. Detection

Global

Issue_01	Governance Issue
Severity	Governance
Description	The architecture exposes multiple governance issues which range from a proxy pattern over privileged minters and multiple other sensitive areas. If one of these addresses get compromised, a total loss of funds is possible
Recommendations	Consider implementing a strong multisig setup for all privileged addresses in the system.
Comments / Resolution	Acknowledged.

Issue_02	Function <code>initialize</code> can be frontrun to hijack deployments.
Severity	Medium
Description	In the <code>deploy_lisaster_test.sol</code> script, all the in-scope contracts are first deployed and subsequently initialized. This makes the <code>initialize</code> functions frontrunnable since they occur as separate transactions, allowing anyone to DOS/hijack deployments with invalid configuration variables. Please note that this issue is only rated as medium because the script currently points to the BSC testnet: <pre>require(block.chainid == 97, "expect BSC testnet [chainId 97]");</pre> The BSC mainnet deployment should be carefully assessed.
Recommendations	Consider using atomic deployment for the BSC mainnet contracts.
Comments / Resolution	Resolved. A new deployment script will be developed for mainnet purposes and this will include atomic deployment + initialization.

LisAster

The **LisAster** contract is a simple ERC20 token implementation which is used within this architecture as a liquid staking receipt for the Aster token and is solely minted by the AsterVault when users deposit the Aster token which is then deposited into the Astherus vault.

Privileged Functions

- mint
- burn

Core Invariants:

INV 1: LisAster.totalSupply == sum of AsterVault.deposit

INV 2: Only AsterVault can mint and burn

Issue_03	Governance Issue: Privileged Minting
Severity	Governance
Description	Currently, the contract exposes one or more governance privileges which can result in unexpected operations and potential loss of funds. For example, a compromised minter address can mint tokens and/or burn tokens from addresses.
Recommendations	Consider implementing a strong multisig setup for all privileged addresses in the system.
Comments / Resolution	Acknowledged.

LisAsterStaking

The **LisAsterStaking** contract is a simple staking contract where users can stake and unstake their **LisAster** tokens. Rewards are currently not incorporated on-chain but the NATSPEC indicates that this is handled via an off-chain system.

Privileged Functions

- pause
- unpause
- emergencyWithdraw

Issue_04	No minimum stake enforcement can be exploited to trick off-chain reward mechanism
Severity	High
Description	<code>LisAsterStaking</code> is the protocol's on-chain staking ledger for <code>lisAster</code>. Users can stake <code>lisAster</code> through <code>stake(amount)</code>, or another caller can stake <code>lisAster</code> for a receiver through <code>stakeFor(receiver, amount)</code>. Both paths call the same internal staking logic, which transfers <code>lisAster</code> into the staking contract and immediately increments <code>balanceOf(receiver)</code> and <code>totalSupply</code>. Users can later call <code>unstake(amount)</code>, which immediately decrements <code>balanceOf(msg.sender)</code> and <code>totalSupply</code>, then transfers the same amount of <code>lisAster</code> back to the user. The staking contract does not contain a reward ledger, reward index, epoch state, lock period, staking timestamp, last-update timestamp, or minimum-holding-duration field. The reward allocation is intentionally performed off-chain. The contract documentation states that the off-chain backend reads <code>balanceOf</code> on an hourly cadence and aggregates the Merkle tree. The resulting cumulative reward amounts are later submitted to <code>LisAsterDistributor</code> through the Merkle root lifecycle. Once a root is accepted, users claim rewards through <code>claim()</code> or <code>claimAndStake()</code>, and the Distributor only verifies the Merkle proof and cumulative amount. It does not verify how long the user was staked during the reward period. The root cause is that reward eligibility is based on a mutable instantaneous staking balance, while the on-chain staking contract does not enforce any minimum staking time or time-weighted participation. A user can increase their <code>LisAsterStaking.balanceOf</code> immediately before the off-chain backend reads staking balances, then decrease it immediately afterward by calling <code>unstake()</code>. If the backend uses that hourly point-in-time balance as the basis for reward allocation, the user can be treated as if they were staked for

	the relevant reward interval even though their actual exposure duration was only a small fraction of the interval. This creates a mismatch between the intended reward model and the enforced on-chain state. The protocol appears to intend that staking balance represents ongoing participation, but the contract only records the current live balance. No on-chain state distinguishes a user who staked for the full hour from a user who staked only around the backend read. The issue becomes economically stronger if the backend snapshot timing is predictable or can be inferred. In that case, a user can automate staking around the expected read, then immediately unstake after the balance is observed. If external lisAster liquidity exists, the user may also exit the lisAster position after unstaking, reducing market exposure to a very short window. That external market-exit assumption is not enforced or provided by the in-scope contracts, but it is a realistic economic amplifier if lisAster is liquid.
Recommendations	A clean fix is not straightforward and depends on how the off-chain aggregator uses <code>balanceOf</code>. Consider introducing a time-weighted approach for balance tracking instead of increasing <code>balanceOf</code> instantly. Alternatively, it might be a possible mitigation to use a higher cadence for balance fetching which makes such an attack less likely. For example a cadence of fetching the balance every 10 minutes would then result in an attacker to set up a bot which sandwiches the balance fetching every 10 minutes and it would encounter a swap-fee every time doing so, which would then eventually be -EV for an attacker. In this scenario, this issue can be considered as partially resolved.
Comments / Resolution	Resolved, the <code>balanceOf</code> fetching mechanism is now randomized which means there is no possibility for an attacker to frontrun a known interval.

Issue_05	<code>stakeFor</code> can be used to bloat merkle tree
Severity	Informational
Description	As per documentation, rewards are allocated to addresses with a stake via off-chain mechanisms. This can be abused by an attacker that deposits 1 wei to thousands of addresses, as it will then increase the leaf amount for the merkle tree unnecessary. Impact 1: Merkle tree size can be unnecessarily increased. An attacker can create many positive staking balances by calling <code>stakeFor()</code> with dust amounts and different receivers. If the backend includes all positive balances, the number of Merkle leaves grows unnecessarily. Impact 2: Off-chain reward processing becomes more expensive and noisy. The backend may need to track, snapshot, calculate, and generate proofs for many dust-funded addresses that do not represent meaningful staking participation. Impact 3: Proof generation and distribution overhead can increase. More Merkle leaves can increase the operational burden of generating proofs, storing proofs, publishing proof data, and supporting users who need claim data. Impact 4: Cumulative roots can inherit persistent dust recipients. If dust recipients receive even a tiny nonzero entitlement, the cumulative model may require those accounts to be carried forward in future roots to preserve historical claimability.
Recommendations	Consider thinking about whether it makes sense to implement a minimum stake amount. This will also impact the <code>claimAndStake</code> control-flow. The most defensive solution would be to completely remove the <code>stakeFor</code> (and <code>claimAndStake</code>) functions as this would then also

	solve other issues.
Comments / Resolution	Acknowledged.

Issue_06	Lack of explicit error enforcement when withdrawing more than owned balance
Severity	Informational
Description	During the <code>unstake</code> function, it will explicitly revert due to underflow if a user attempts to withdraw more than the owned balance. While this is fine, it means there is no explicit revert message which can be confusing during tracing processes.
Recommendations	Consider acknowledging this issue.
Comments / Resolution	Resolved.

Issue_07	Violation of CEI within <code>_stake</code>
Severity	Informational
Description	The <code>_stake</code> function executes external calls before storage adjustments which violates the checks-effects-interactions pattern and can become a precondition for reentrancy vulnerabilities.
Recommendations	Consider adjusting the storage before the external call.
Comments / Resolution	Resolved.

Issue_08	<code>emergencyWithdraw</code> will desync <code>totalSupply</code>
Severity	Informational
Description	<code>LisAsterStaking</code> is the protocol's staking ledger for <code>lisAster</code>. Users deposit <code>lisAster</code> through <code>stake{amount}</code>, while third parties can deposit <code>lisAster</code> for another receiver through <code>stakeFor{receiver, amount}</code>. Both flows transfer <code>lisAster</code> into the staking contract and increase the receiver's internal staking balance through <code>balanceOf{receiver}</code>. The contract also increases the global <code>totalSupply</code> value, which represents the aggregate amount of <code>lisAster</code> that users are entitled to withdraw from the staking contract. Users exit through <code>unstake{amount}</code>. This decreases <code>balanceOf[msg.sender]</code> and <code>totalSupply</code>, then transfers <code>lisAster</code> from the staking contract back to the user. Therefore, the <code>unstake</code> path assumes that the actual <code>lisAster</code> balance held by the staking contract is sufficient to cover the internal staking ledger. The staking contract also exposes <code>emergencyWithdraw{token, amount}</code> to the <code>MANAGER</code> role. This function transfers the requested token amount to the manager caller. It can be used for any token address, including the configured <code>lisAster</code>. When used on <code>lisAster</code>, it transfers tokens out of the same pool that backs users' staking balances. The root cause is that <code>emergencyWithdraw()</code> can remove <code>lisAster</code> from the staking contract without updating the staking accounting ledger. The staking ledger has two relevant accounting surfaces: user-level balances tracked in <code>balanceOf{user}</code>, and the global staking liability tracked in <code>totalSupply</code>. These values are increased when users stake and decreased only when users unstake. <code>emergencyWithdraw()</code> does not modify either value. A similar issue exists within the <code>LisAsterDistributor</code> contract.

Recommendations	Consider acknowledging this issue and use emergencyWithdraw only in emergency situations.
Comments / Resolution	Acknowledged.

AsterVault

The **AsterVault** contract allows users to deposit their Aster tokens in exchange for **lisAster** tokens. The Aster tokens are then deposited into the **AstherusVault** via the **depositFor** function selector which is expected to mint tokens on the Aster chain to the **LisAsterManager** address.

Appendix: Astherus Interaction

AstherusVault is a BSC-side custody contract operated by Astherus that accepts ERC20/native deposits and emits a **Deposit[forAddress, currency, isNative, amount, broker]** event.

Astherus's off-chain backend reads that event and credits **forAddress** with an equivalent balance on Aster Chain.

Withdrawals back out require either a signature from Astherus's signer (with an hourly USD limit gate) or a $\geq 2/3$ power-weighted multisig from a registered validator set.

It also supports gasless deposits via permit, fee accounting, and role-gated administration, the contract is UUPS-upgradeable behind a timelock.

In this protocol's flow, AsterVault only uses its **depositFor** entry point to push ASTER in on behalf of the **lisAsterManager** EOA, every other **AstherusVault** feature is unused by Lista's in-scope contracts.

Privileged Functions

- **setBroker**
- **setMinDeposit**
- **pause**
- **unpause**

Core Invariants:

INV 1: User must receive same amount of **lisAster** as deposited aster

INV 2: A deposit must be $\geq$ **minDeposit**

Issue_09	Immutable lisAsterManager contract may become problematic
Severity	Medium
Description	Currently, the lisAsterManager is immutable and it is the EOA where the tokens on the DST chain will be minted to. This can become problematic because if this address is compromised, there is no setter for changing it. While it indeed can be paused and as such minting on the DST will be paused to at least prevent damage occurring from normal business operations, there is a specific way how this can be efficiently attacked: An attacker can deliberately deposit Aster to receive lisAster just to regain the Aster on the DST chain if he has compromised the key, via a recursive exploit and as such, if the lisAsterManager is compromised, an attacker can mint large amounts of lisAster via this repetitive attack.
Recommendations	Consider implementing a setter function for this address.
Comments / Resolution	Resolved.

Issue_10	LisAster 1:1 peg does not hold in case lisAsterManager incurs losses on the Aster chain
Severity	Low
Description	LisAster is supposed to be the 1:1 LST of ASTER deposited in the Astherus vault. When a deposit occurs from the AsterVault to the Astherus vault, Astherus's backend syncs the credited balance to the same EOA [the lisAsterManager] on Aster Chain within 1-3 minutes, where the manager uses it for yield accrual. However, if the manager incurs losses on Aster, the 1:1 peg is no longer materially held, even though this may not surface as user-facing insolvency on the AsterVault, given the absence of a redemption path. This would also create systemic mispricing affecting external integrations pricing LisAster at par e.g. a lending market treating lisAster as collateral or a DEX liquidity pool.
Recommendations	Consider documenting this risk and ensuring the peg is maintained by the MANAGER in case losses are incurred.
Comments / Resolution	Acknowledged.

Issue_11	Lack of support for tx-tax tokens
Severity	Informational
Description	Currently, this contract has no support for tx-tax tokens. While that is no issue related to the Aster token, it is notable that the AstherusVault implements such logic.
Recommendations	Consider acknowledging this issue.
Comments / Resolution	Acknowledged.

Issue_12	Synchronous lisAster minting while bridge mechanism is asynchronous
Severity	Informational
Description	AsterVault is the sole in-scope minting gateway for lisAster. Users deposit ASTER into AsterVault.deposit(amount, receiver), after which the contract transfers ASTER from the user, approves the external AstherusVault, and calls IAstherusVault.depositFor(asterToken, lisAsterManager, amount, broker). The lisAsterManager is the Lista-operated EOA that should receive the Astherus-side credit, and the contract comments state that Astherus's backend syncs the credited balance to the same EOA on Aster Chain within one to three minutes. After the external depositFor() call returns successfully, AsterVault immediately mints lisAster 1:1 to the user-selected receiver. This means the local BSC-side lisAster mint is synchronous, while the Astherus-side backend sync to Aster Chain is asynchronous and external to the in-scope Solidity code. The same AsterVault.deposit() path is also used by LisAsterRewards.notifyRewards(). In that flow, the MANAGER provides ASTER rewards, the Rewards contract optionally deducts a fee, then redeposits the net ASTER through AsterVault.deposit(net, address[this]). This mints lisAster to LisAsterRewards, which can later be forwarded to LisAsterDistributor and allocated to users through cumulative Merkle roots. The root cause is that AsterVault.deposit() treats successful return from AstherusVault.depositFor() as sufficient to mint lisAster, even though the architecture describes the Astherus-side Aster Chain credit as an asynchronous backend sync that happens after the BSC deposit. The contract does not verify that the backend sync actually completed, that the Aster Chain balance became visible, that the credited EOA is operational, or that the external ledger state matches the locally minted lisAster amount. There is no local state

	tracking “pending external sync,” no delayed mint, no reconciliation flag, and no post-sync confirmation before lisAster becomes usable. As a result, the user receives liquid lisAster immediately after <code>depositFor()</code> returns. The user can transfer it, stake it through <code>LisAsterStaking</code>, or indirectly participate in future reward allocation before the external Astherus-side sync is confirmed. Downstream contracts treat the minted lisAster as fully valid because they only consume ERC20 balances and local accounting state. This creates a mismatch between the local lisAster supply state and the external Astherus operational state. If the backend sync fails, is delayed beyond the expected window, credits the wrong account, or becomes unavailable for the <code>lisAsterManager</code>, the protocol has already issued lisAster to the user despite the external credit not being available as expected on Aster Chain.
Recommendations	Consider acknowledging this issue.
Comments / Resolution	Acknowledged.

Issue_13	<code>AsterVault.initialize()</code> allows <code>minDeposit</code> to be zero while <code>setMinDeposit</code> forbids it
Severity	Informational
Description	The <code>minDeposit_</code> parameter has no zero-value check, while <code>setMinDeposit</code> explicitly rejects zero. The two paths therefore enforce different invariants: a fresh deployment can be initialized with <code>minDeposit = 0</code> even though the post-deploy setter would not allow re-setting to the same value.
Recommendations	Add <code>require(minDeposit_ > 0)</code> to <code>AsterVault.initialize()</code> to mirror the post-deploy setter.
Comments / Resolution	Resolved.

Issue_14	Missing <code>LisAster</code> redemption path
Severity	Informational
Description	The <code>AsterVault</code> provides users with a way to deposit Aster to mint <code>LisAster</code>. However no mechanism exists that allows the user to redeem the <code>LisAster</code> for Aster again. This could create a situation where if the yield received from deposits is not attractive and <code>LisAster</code> does not have external markets with sufficient liquidity, users could be disincentivized from depositing in the <code>AsterVault</code>.
Recommendations	Consider documenting if intended behaviour or implement a redemption path for users.
Comments / Resolution	Acknowledged.

LisAsterRewards

The **LisAsterRewards** contract allows a privileged address to deposit Aster tokens which is then converted via the AsterVault to LisAster. Any idle LisAster can then be distributed by the BOT to the LisAsterDistributor contract where users can eventually claim rewards via a cumulative fashion.

Privileged Functions

- `setDistributor`
- `setFeeReceiver`
- `setFeeRate`
- `notifyRewards`
- `distributeRewards`
- `pause`
- `unpause`

Core Invariants:

INV 1: `notifyRewards` must always receive “net” amount of `lisAster`

Issue_15	Fees can be bypassed via direct lisAster transfer
Severity	Low
Description	<code>LisAsterRewards</code> is the protocol's reward conversion and staging contract. In the intended flow, the <code>MANAGER</code> calls <code>notifyRewards(amount)</code> with <code>ASTER</code>. The contract pulls <code>ASTER</code> from the manager, optionally deducts an <code>ASTER</code> fee based on <code>feeRate</code> and <code>feeReceiver</code>, then deposits the net <code>ASTER</code> amount through <code>AsterVault.deposit(net, address[this])</code>. <code>AsterVault</code> forwards the <code>ASTER</code> to the external <code>Astherus</code> vault and mints <code>lisAster</code> 1:1 to <code>LisAsterRewards</code>. The Rewards contract then verifies that the amount of <code>lisAster</code> received equals the net <code>ASTER</code> amount and emits <code>RewardsNotified</code>. After <code>lisAster</code> has accumulated in <code>LisAsterRewards</code>, the <code>BOT</code> can call <code>distributeRewards(amount)</code>. This function only checks that <code>amount</code> is nonzero, that the Distributor is configured, and that the Rewards contract currently holds at least <code>amount</code> <code>lisAster</code>. It then approves the Distributor and calls <code>LisAsterDistributor.notifyRewards(amount)</code>. The Distributor pulls <code>lisAster</code> from Rewards and increments <code>totalNotified</code> by the distributed amount. The important architectural detail is that <code>distributeRewards()</code> is balance-based. It does not distinguish <code>lisAster</code> minted by the current reward notification flow from <code>lisAster</code> that entered the Rewards contract through any other route. The root cause is that <code>LisAsterRewards.distributeRewards()</code> treats the Rewards contract's current <code>lisAster</code> balance as distributable reward inventory, regardless of how that balance was created. The intended reward path is <code>ASTER</code>-based. <code>ASTER</code> enters through <code>notifyRewards()</code>, fee logic is applied, the net amount is deposited through <code>AsterVault</code>, and <code>lisAster</code> is minted to the Rewards contract. This path creates a clear relationship between the gross <code>ASTER</code> reward, the fee amount, the net minted <code>lisAster</code> amount, and the later distributed <code>lisAster</code> amount.

	However, <code>lisAster</code> can also enter <code>LisAsterRewards</code> without calling <code>notifyRewards()</code>. For example, an external holder can first obtain <code>lisAster</code> via the vault and then transfer <code>lisAster</code> directly to the Rewards contract, or a user can call <code>AsterVault.deposit(amount, address[LisAsterRewards])</code>, causing freshly minted <code>lisAster</code> to be sent directly to Rewards as the receiver. In both cases, the Rewards contract's <code>lisAster</code> balance increases, but the <code>notifyRewards()</code> flow is not executed for that balance. No ASTER fee is applied, no <code>RewardsNotified</code> event is emitted for that <code>lisAster</code>, and no local reward-cycle provenance is stored. If the BOT later calls <code>distributeRewards(amount)</code>, the donated or externally sourced <code>lisAster</code> can be forwarded to the Distributor. The Distributor then increments <code>totalNotified</code>, making the amount eligible for future Merkle allocation. From the Distributor's perspective, this looks like normal notified reward funding, even though the <code>lisAster</code> did not pass through the intended Rewards notification path. This does not automatically inflate <code>lisAster</code> supply. Donated <code>lisAster</code> already exists. The issue is that the protocol's reward accounting and fee model can be bypassed or made ambiguous because distributed rewards are not provenance-checked.
Recommendations	Consider acknowledging this issue and enforce indirectly that the BOT always walks through the <code>notifyRewards</code> function. Alternatively, if such an enforcement cannot be guaranteed, it might be an idea to implement a local tracker for the received <code>lisAster</code> amount via <code>notifyRewards</code>, which is then used to determine the reward amount during <code>distributeRewards</code> and is also likewise decreasing during the same. However, we do not see this as required as the BOT is likely a protocol controlled address with the intention to pay the fees.
Comments / Resolution	Acknowledged.

Issue_16	<code>notifyRewards</code> can revert when the fee drops net below <code>AsterVault.minDeposit()</code>
Severity	Informational
Description	<code>LisAsterRewards.notifyRewards()</code> pulls <code>amount</code> of ASTER, deducts a fee, and forwards the net to <code>AsterVault.deposit()</code> . This function requires <code>amount >= minDeposit</code> . When <code>MANAGER</code> pushes an amount close to <code>minDeposit</code> and a non-zero fee is configured, <code>net</code> can fall below <code>minDeposit</code> and the call reverts. The <code>minDeposit</code> check itself is a Lista-side defensive measure to discourage dust deposits and. <code>AstherusVault.depositFor()</code> only requires <code>amount > 0</code> .
Recommendations	Consider acknowledging this issue
Comments / Resolution	Acknowledged.

Issue_17	Fee calculation in <code>notifyRewards()</code> rounds down
Severity	Informational
Description	In <code>LisAsterRewards.notifyRewards()</code>, the fee is computed as <code>fee = [amount * feeRate] / PRECISION</code>. Solidity integer division truncates toward zero, so any sub-unit remainder is dropped from the fee and remains in the net amount that gets forwarded to <code>AsterVault.deposit()</code> and minted as <code>lisAster</code> to stakers. For small reward batches the rounding can fully zero out the fee. It does mean that the realized effective fee rate is slightly less than the nominal <code>feeRate</code>, and that very small reward batches escape fees entirely.
Recommendations	Consider acknowledging this issue.
Comments / Resolution	Acknowledged.

LisAsterDistributor

LisAsterDistributor is a cumulative Merkle distributor for lisAster rewards. Inflows arrive via `notifyRewards` (callable only by the `LisAsterRewards` contract, which pulls `lisAster` in and increments `totalNotified`).

Allocations are encoded off-chain as a Merkle tree whose leaves are `keccak256[abi.encode(block.chainid, account, lisAster, cumulativeAmount)]`, where `cumulativeAmount` is the account's lifetime entitlement, the root is rotated through a two-step time-locked flow whereas BOT calls `setPendingMerkleRoot(root, totalAllocated)`, then `acceptMerkleRoot` after `waitingPeriod ≥ MIN_WAITING_PERIOD = 6 hours`, while MANAGER retains a veto via `revokePendingMerkleRoot` during the wait.

Users claim deltas via `claim(account, cumulativeAmount, proof)` or `claimAndStake(cumulativeAmount, proof)`.

Internal `_consume` computes `payable = cumulativeAmount – claimed[account]`, updates `claimed[account]` to the new cumulative, and bumps `totalClaimed`.

Privileged Functions

- `notifyRewards`
- `setPendingMerkleRoot`
- `acceptMerkleRoot`
- `revokePendingMerkleRoot`
- `changeWaitingPeriod`
- `pause`
- `unpause`
- `emergencyWithdraw`

Core Invariants:

INV 1: `totalNotified` must be $\geq$ `newTotalAllocated`

INV 2: `newTotalAllocated` must be $\geq$ `totalAllocated`

INV 3: `pendingMerkleRoot/pendingTotalAllocated` and `lastSetTime` must be reset upon

INV 4: `acceptMerkleRoot` and `revokePendingMerkleRoot`

INV 5: `totalAllocated` must match the sum of user's `cumulativeAmounts`

INV 6: Rewards must only stem from `LisAsterRewards` deposits

INV 7: Each user's leaf.cumulativeAmount must be >= than leafBefore.cumulativeAmount

INV 8: merkleRoot setting must always walk through pending/acceptance flow

INV 9: Any claim must consume leaf.cumulativeAmount - alreadyClaimed

Issue_18	<code>acceptMerkleRoot()</code> is callable only by BOT creating a single point of failure for reward distribution
Severity	Governance
Description	In <code>LisAsterDistributor.acceptMerkleRoot()</code>, the function is gated by <code>onlyRole(BOT)</code>. Both <code>setPendingMerkleRoot</code> and <code>acceptMerkleRoot</code> require the same role. If the BOT key becomes unavailable for any reason, no party can finalize a staged Merkle root. The 6-hour waiting period passes and the pending root remains uncalleable. Reward distribution stalls indefinitely until <code>DEFAULT_ADMIN</code> grants the BOT role to a new EOA and that EOA stages plus accepts a new root. During the stall, <code>lisAster</code> pushed into the <code>Distributor</code> via <code>LisAsterRewards.distributeRewards()</code> accumulates with no claim path. Allowing <code>MANAGER</code> or <code>DEFAULT_ADMIN</code> to also call <code>acceptMerkleRoot()</code> once the wait period has elapsed would eliminate the single point of failure without weakening the security model.
Recommendations	Consider allowing additional roles to call <code>acceptMerkleRoot()</code> once the wait period has elapsed.
Comments / Resolution	Acknowledged.

Issue_19	<code>changeWaitingPeriod()</code> lacks an upper bound allowing admin to stall reward distribution indefinitely
Severity	Governance
Description	The new <code>waitingPeriod</code> value is validated only against a floor [<code>newWaitingPeriod >= MIN_WAITING_PERIOD</code>] and is not bounded above. <code>DEFAULT_ADMIN</code> can set <code>waitingPeriod</code> to any <code>uint256</code> value, including ones that effectively make <code>acceptMerkleRoot()</code> uncallable forever (for example 100 years, or <code>type(uint256).max</code> which overflows in the <code>lastSetTime + waitingPeriod</code> arithmetic and reverts on every <code>acceptMerkleRoot()</code> call).
Recommendations	Introduce a <code>MAX_WAITING_PERIOD</code> constant and validate against it in both <code>changeWaitingPeriod()</code> and the initializer, mirroring the floor check that already exists.
Comments / Resolution	Acknowledged.

Issue_20	Lack of incorporation for address[this] allows for replay-attacks in same-chain contracts
Severity	Low
Description	The leaf preimage encodes only block.chainid, account, the reward token address lisAster, and cumulativeAmount. It does not bind the leaf to the specific Distributor instance, address[this] is absent from the preimage. The chain-id binding closes the cross-chain replay surface and the token-address binding differentiates leaves intended for distinct reward tokens, but two Distributor contracts deployed on the same chain that both reward lisAster and that hold the same merkleRoot will produce byte-identical leaf hashes for any given {account, cumulativeAmount} pair. A proof valid against one root therefore verifies against the other. The root cause is that the leaf domain does not include a verifying-contract identifier [the Merkle-proof analogue of EIP-712's verifyingContract], so the Merkle commitment scheme is not bound to the instance that consumes it. If a second LisAsterDistributor [or any contract using the same leaf encoding] is ever deployed on the same chain and ever holds the same merkleRoot as the first, then, every account with a non-zero leaf in that shared root can redeem the same cumulativeAmount independently on each instance, because each instance maintains its own claimed[account] counter starting from zero.
Recommendations	Consider incorporating address[this] in the leaf.
Comments / Resolution	Acknowledged.

Issue_21	Trigger of taxable events due to claiming on behalf
Severity	Low
Description	In certain jurisdictions, a taxable event will be triggered as soon as an individual receives tokens (airdrop, etc). This can be done deliberately via the claiming on behalf, which can then trigger a taxable event which is undesired by the recipient.
Recommendations	Consider acknowledging this issue or removing the claiming on behalf.
Comments / Resolution	Acknowledged.

Issue_22	Lack of on-chain enforcement that <code>newTotalAllocated</code> matches sum of leaf amounts
Severity	Low
Description	Currently, there is no such enforcement that the <code>newTotalAllocated/totalAllocated</code> value matches the sum of all leaves for the corresponding merkle tree, which could then result in insolvency if all users claim and the aggregate of the cumulative values is larger. It can then happen that all funds in the contract can be drained either accidentally or intentionally via a compromised BOT role.
Recommendations	Consider making sure this is always properly enforced during the merkle tree creation. Furthermore, as additional safeguard, we recommend a check that claims do not exceed the <code>totalAllocated</code> value.
Comments / Resolution	Resolved.

Issue_23	Change of <code>waitingPeriod</code> will be immediately reflected on pending root
Severity	Low
Description	LisAsterDistributor rotates the live Merkle root through a two-step time-locked flow. BOT stages a candidate root via <code>setPendingMerkleRoot(root, newTotalAllocated)</code>, which records <code>lastSetTime = block.timestamp</code>, and then promotes it via <code>acceptMerkleRoot()</code> once <code>block.timestamp >= lastSetTime + waitingPeriod</code>. The MANAGER role holds an asymmetric escape hatch: <code>revokePendingMerkleRoot()</code> which can discard the staged root at any time during the wait. The constant <code>MIN_WAITING_PERIOD = 6 hours</code> floors the configurable <code>waitingPeriod</code> storage slot. Both the constructor-time initialize check and the admin setter <code>changeWaitingPeriod(newWaitingPeriod)</code> enforce this floor, and the contract's inline documentation explicitly frames the floor as a guarantee that MANAGER always has at least six hours of reaction time against a malicious or erroneous root staged by BOT. <code>acceptMerkleRoot</code> evaluates the time-lock against the live <code>waitingPeriod</code> storage slot rather than a value snapshotted at stage time. <code>setPendingMerkleRoot</code> only persists <code>lastSetTime</code>, never copying or pinning the wait duration that was in effect when the candidate was staged. Because <code>changeWaitingPeriod</code> mutates that same live slot without any check on whether a pending root currently exists, lowering <code>waitingPeriod</code> while a stage is in flight retroactively shortens or instantly eliminates the unlock interval for that already-staged root.
Recommendations	Consider ensuring that the <code>waitingPeriod</code> cannot be altered as long as <code>pendingMerkleRoot</code> is non-zero.
Comments / Resolution	Resolved. Something to note here however is that the BOT can DOS the DEFAULT_ADMIN_ROLE from changing the waiting period by frontrun setting a new pending merkle root. Just a risk which can be acknowledged since the admin can simply remove the BOT role, revoke the root and change the waiting period.

Issue_24	Minimum period might be too high for short cadence stagings
Severity	Low
Description	<code>LisAsterDistributor</code> rotates its Merkle root through a two-step time-locked flow. BOT stages a candidate root with <code>setPendingMerkleRoot[root, newTotalAllocated]</code>, which records <code>pendingMerkleRoot</code>, <code>pendingTotalAllocated</code>, and <code>lastSetTime = block.timestamp</code>. The candidate becomes promotable once <code>block.timestamp >= lastSetTime + waitingPeriod</code>, at which point BOT calls <code>acceptMerkleRoot()</code> to make it live. MANAGER retains a veto via <code>revokePendingMerkleRoot()</code> during the wait. The off-chain backend which is the source of the candidate roots is documented as reading <code>LisAsterStaking.balanceOf</code> on an hourly cadence to aggregate the per-account stake-weights that drive each tree, so the natural publication rhythm of the backend *could be* faster than the on-chain <code>waitingPeriod ≥ MIN_WAITING_PERIOD = 6 hours</code> lock. The design intent of the lock is to give MANAGER a guaranteed minimum window in which to revoke a wrong root. <code>setPendingMerkleRoot</code> is permissively idempotent on the pending slot: it allows BOT to overwrite an existing <code>pendingMerkleRoot</code> at any moment, and the act of overwriting deterministically resets <code>lastSetTime</code> to the current block timestamp. There is no separate "replace pending" path, no event differentiating an initial stage from a re-stage beyond timestamping, and no check that the existing pending has been either accepted or revoked before a new one is recorded. Because <code>acceptMerkleRoot</code> reads <code>lastSetTime</code> directly when evaluating its time-lock gate, any re-stage shifts the unlock boundary forward by exactly the elapsed time since the prior stage. If BOT's effective staging cadence is $\Delta < \text{waitingPeriod}$, every successive stage occurs before the lock matures, the previous root and its accumulated wait are silently discarded, and the unlock condition <code>block.timestamp >= lastSetTime + waitingPeriod</code> is never satisfied at the moment of any subsequent <code>setPendingMerkleRoot</code>

	call. The root cause is the *potential* asymmetry between the off-chain backend's natural rhythm and the on-chain lock floor, exposed by the lack of any contract-level enforcement that staging cadence respect waitingPeriod .
Recommendations	Consider thinking about whether it can happen that the merkle root staging mechanism can occur more frequently than the minimum period of 6 hours. If that can ever happen, the minimum period should be removed.
Comments / Resolution	Acknowledged.

Issue_25	totalAllocated check does not guarantee per-leaf correctness
Severity	Informational
Description	<code>LisAsterDistributor</code> implements a cumulative-style Merkle distribution: each leaf encodes an account's lifetime entitlement <code>cumulativeAmount</code>, the contract tracks per-account lifetime redemption in <code>claimed[account]</code>, and <code>_consume</code> pays out the delta. Roots rotate through a two-step time-locked flow, <code>BOT.setPendingMerkleRoot(root, newTotalAllocated)</code> followed by <code>BOT.acceptMerkleRoot()</code> after <code>waitingPeriod</code>. The off-chain backend is therefore expected to publish each successive root as a strictly-non-decreasing snapshot of total entitlements, with the per-leaf entry for any given account expected (but not enforced) to be at least the value that the same account carried in the previous root, so that each redemption is the "delta since last root" rather than a possibly-negative diff against a regression. The contract enforces monotonicity only on the sum of leaves (<code>newTotalAllocated >= totalAllocated</code>) and only on the gap between any leaf and the same account's prior lifetime claim (<code>cumulativeAmount > claimed[account]</code> inside <code>_consume</code>). It enforces nothing on the per-leaf, per-account relationship between successive roots. Consequently, the backend can publish a root in which user A's <code>cumulativeAmount</code> is reduced (including to zero) so long as the deducted amount is reassigned to one or more other leaves to keep the aggregate non-decreasing. The structurally-forced reallocation pattern combines adversely with the <code>waitingPeriod</code> mechanic: while the candidate root sits in <code>pendingMerkleRoot</code>, the previously-accepted root is still live, so account A, whose entitlement is about to be reduced, retains the ability to redeem the old leaf via <code>claim</code> or <code>claimAndStake</code> for the duration of the wait. If A redeems before <code>acceptMerkleRoot</code>, A receives the full old <code>cumulativeAmount - claimed[A]</code>, and once the new root is accepted, the recipients of the reallocated amount are also able to redeem their newly-increased

	entitlements against the same lisAster pool. The root cause is the absence of a leaf-level invariant tying each account's new cumulativeAmount to the maximum of [prior root's cumulative for that account, claimed[account]], which the contract cannot natively enforce without inspecting every prior leaf but which is also not enforced by any off-chain commitment that the contract verifies.
Recommendations	Consider acknowledging this issue.
Comments / Resolution	Acknowledged.

Issue_26	Permissionless claim can be used to grief <code>claimAndStake</code>
Severity	Informational
Description	<code>LisAsterDistributor</code> supports two claim paths for <code>lisAster</code> rewards. The first path is <code>claim(account, cumulativeAmount, proof)</code>, which is permissionless and allows any caller to trigger a claim for any nonzero account. The Merkle leaf binds the payout recipient to <code>account</code>, and the Distributor transfers the claimed <code>lisAster</code> to that same account. Therefore, this route cannot be used to redirect another user's rewards to the caller. The second path is <code>claimAndStake(cumulativeAmount, proof)</code>, which is self-only and implicitly uses <code>msg.sender</code> as the claimant. This route consumes the user's reward entitlement, approves the configured <code>LisAsterStaking</code> contract, and calls <code>stakeFor(msg.sender, payableAmount)</code>. As a result, the claimed <code>lisAster</code> does not go to the user's wallet. Instead, it is atomically moved into the staking contract and credited to <code>LisAsterStaking.balanceOf(msg.sender)</code>. Both claim paths share the same internal <code>_consume()</code> logic. <code>_consume()</code> verifies the current Merkle root, calculates <code>payableAmount = cumulativeAmount - claimed[account]</code>, updates <code>claimed[account]</code> to <code>cumulativeAmount</code>, and increments <code>totalClaimed</code>. This means the user's claim entitlement is consumed once, regardless of whether the wallet-claim route or the claim-and-stake route was used. The root cause is that <code>claim()</code> allows third parties to consume a user's cumulative reward entitlement through the wallet route, while <code>claimAndStake()</code> consumes the same entitlement through the staking route. The two routes have the same economic source, but they leave different downstream state. If a user intends to call <code>claimAndStake()</code>, the user expects the claimed reward to immediately increase their staking balance. However, because <code>claim(account, cumulativeAmount, proof)</code> is permissionless, another caller can submit the user's valid proof first.

	This updates <code>claimed[user]</code> and transfers the <code>lisAster</code> to the user's wallet. After that, the user's <code>claimAndStake()</code> call for the same <code>cumulativeAmount</code> has no remaining claimable delta. <code>_consume()</code> sees that <code>claimed[user]</code> is already equal to the submitted <code>cumulativeAmount</code> and reverts with "nothing to claim". The user still receives the <code>lisAster</code>, so this is not a theft issue. The issue is that the third party can force the reward to settle through a different protocol route than the user intended. That matters because only <code>claimAndStake()</code> updates the staking ledger atomically. A wallet claim does not increase <code>LisAsterStaking.balanceOf[user]</code>.
Recommendations	Consider acknowledging this issue.
Comments / Resolution	Acknowledged.

Issue_27	Multiple leaves per account are not considered
Severity	Informational
Description	<code>LisAsterDistributor</code> distributes <code>lisAster</code> rewards through cumulative Merkle roots. The Distributor stores a single live <code>merkleRoot</code>, a global <code>totalAllocated</code>, and per-account claim progress in <code>claimed[account]</code>. Users can claim rewards through <code>claim(account, cumulativeAmount, proof)</code>, or they can use <code>claimAndStake(cumulativeAmount, proof)</code> to claim and immediately stake the reward through <code>LisAsterStaking</code>. Both claim paths use the same internal <code>_consume()</code> logic. <code>_consume()</code> verifies that the supplied leaf is part of the current Merkle root, reads the user's previously claimed amount from <code>claimed[account]</code>, computes the payable delta as <code>cumulativeAmount - claimed[account]</code>, updates <code>claimed[account]</code> to the submitted <code>cumulativeAmount</code>, and increments <code>totalClaimed</code> by the payable delta. The Merkle leaf binds the chain id, account, <code>lisAster</code> token address, and cumulative amount. The claim state, however, is keyed only by account. This means the protocol assumes that each accepted root contains at most one effective cumulative reward value per account, and that this value represents the account's full lifetime cumulative entitlement. The root cause is that the cumulative claim cursor is stored only per account, while the Merkle tree itself does not enforce uniqueness of accounts. If the off-chain root generator includes multiple valid leaves for the same account, the contract does not treat those leaves as separate reward entitlements. Instead, all same-account leaves share the same <code>claimed[account]</code> cursor. This creates an issue if the off-chain system builds a tree where same-account leaves are intended to be additive. For example, the backend may calculate one reward component as <code>100</code> and another reward component as <code>200</code>, then insert both as separate leaves for the same account. On-chain, this does not result in a total claimable

	amount of 300. The Distributor interprets the submitted values as alternative cumulative amounts in the same claim stream. If the user first consumes the 100 leaf, <code>claimed[account]</code> becomes 100. When the user later consumes the 200 leaf, the payable amount is only $200 - 100 = 100$. The total received is 200, not 300. If the user consumes the 200 leaf first, <code>claimed[account]</code> becomes 200, and the 100 leaf can no longer be consumed because it is not greater than the already-claimed value. The issue is therefore not that the proof verification is incorrect. The issue is a semantic mismatch between a potentially multi-leaf off-chain reward representation and the on-chain per-account cumulative cursor model. The contract requires the off-chain Merkle generator to merge all reward components for an account into one lifetime cumulative value before root construction.
Recommendations	Consider ensuring that each account will only ever get 1 leaf.
Comments / Resolution	Acknowledged.

Issue_28	Root revokes can occur post waiting period contrary to natspec description
Severity	Informational
Description	The natspec in <code>LisAsterDistributor</code> contract mentions the following: <i>MANAGER holds the veto: it may 'revokePendingMerkleRoot' at any time during the wait</i> However the <code>MANAGER</code> can revoke the root after the waiting period as well in case the <code>BOT</code> has not accepted the root yet.
Recommendations	Consider clarifying the natspec documentation.
Comments / Resolution	Resolved.

Issue_29	<code>emergencyWithdraw</code> can result in insolvency
Severity	Informational
Description	The contract expects (but does not enforce) that the aggregated leaves are equal to <code>totalAllocated</code>. If this assumption holds an <code>emergencyWithdraw</code> is executed, it can happen that the contract becomes insolvent due to the withdrawal, as not the balance is below <code>totalAllocated</code>.
Recommendations	Consider using the <code>emergencyWithdraw</code> only in real emergency situations and acknowledging this issue.
Comments / Resolution	Acknowledged.